Chapter 2

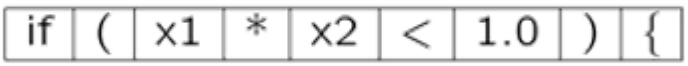
Lexical Analysis

The first phase of the compiler is lexical analysis. The lexical analyzer breaks a sentence into a **sequence of words** or **tokens** and ignores white spaces and comments. It generates a stream of tokens from the input. This is modeled through **regular expressions** and the structure is recognized through **finite state automata**. If the token is not valid i.e., does not fall into any of the identifiable groups, then the lexical analyzer reports an error. Lexical analysis thus involves recognizing the tokens in the source program and reporting errors, if any.

- Recognize tokens and ignore white spaces, comments



Generates token stream



- Error reporting
- Model using regular expressions
- Recognize using Finite State Automata

- Sentences consist of string of tokens (a syntactic category) for example, number, identifier, keyword, string
- Sequences of characters in a token is a **lexeme** for example, 100.01, counter, const, "How are you?"
- Rule of description is a **pattern** for example, letter(letter/digit)*
- Discard whatever does not contribute to parsing like white spaces (blanks, tabs, newlines) and comments
- **construct constants**: convert numbers to token num and pass number as its attribute, for example, integer 31 becomes <num, 31>
- recognize keyword and identifiers for example counter = counter + increment becomes id = id + id /*check if id is a keyword*/

We often use the terms "**token**", "**pattern**" and "**lexeme**" while studying lexical analysis.

Lets see what each term stands for.

Token: A token is a **syntactic category**. Sentences consist of a string of tokens. For example, number, identifier, keyword, string etc. are tokens.

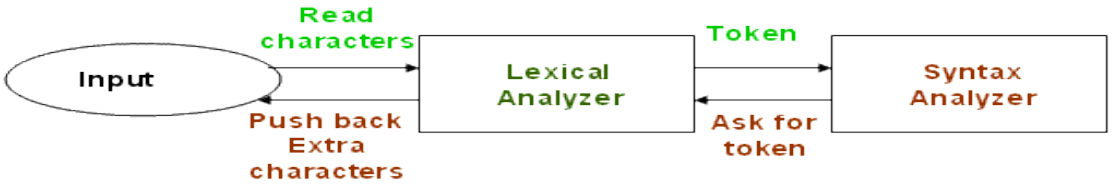
Lexeme: Sequence of characters in a token is a lexeme. For example 100.01, counter, const, "How are you?" etc are lexemes.

Pattern: Rule of description is a pattern. For example letter (letter | digit)* is a pattern to symbolize a set of strings which consist of a letter followed by a letter or digit. In general, there is a set of strings in the input for which the same token is produced as output. This set of strings is described by a rule called a ***pattern*** **associated** with the token. This pattern is said to *match* each string in the set. A ***lexeme*** is a sequence of characters in the source program that is matched by the pattern for a token. The patterns are specified using regular expressions. For example, in the Pascal statement.

Const pi = 3.1416;

The substring **pi** is a lexeme for the token "identifier". We discard whatever does not contribute to parsing like white spaces (blanks, tabs, new lines) and comments. When more than one pattern matches a lexeme, the lexical analyzer must provide additional information about the particular lexeme that matched to the subsequent phases of the compiler. For example, the pattern num matches both 1 and 0 but it is essential for the code generator to know what string was actually matched. The lexical analyzer collects information about tokens into their associated attributes. For example integer 31 becomes <num, 31>. So, the constants are constructed by converting numbers to token 'num' and passing the number as its attribute. Similarly, we recognize keywords and identifiers. For example count = count + inc becomes id = id + id.

Interface to other phases



- Push back is required due to look ahead **for example > = and >**
- It is implemented through a buffer
    - Keep input in a buffer
    - Move pointers over the input

The lexical analyzer reads characters from the input and passes tokens to the syntax analyzer whenever it asks for one. For many source languages, there are occasions when the lexical analyzer needs to look ahead several characters beyond the current lexeme for a pattern before a match can be announced. For example, > and >= cannot be distinguished merely based on the first character >. Hence, there is a need to maintain a buffer of the input for look ahead and push back. We keep the input in a buffer and move pointers over the input. Sometimes, we may also need to push back extra characters due to this look ahead character.

Approaches to implementation

- Use assembly language Most efficient but most difficult to implement
- Use high level languages like C Efficient but difficult to implement
- Use tools like **lex**, **flex** Easy to implement but not as efficient as the first two cases

Lexical analyzers can be implemented using many approaches/techniques:

- **Assembly language** : We have to take input and read it character by character. So we need to have control over low level I/O. Assembly language is the best option for that because it is the most efficient. This implementation produces very efficient lexical analyzers. However, it is most difficult to implement, debug and maintain.

- **High-level language like C**: Here we will have a reasonable control over I/O because of high-level constructs. This approach is efficient but still difficult to implement.
- **Tools like Lexical Generators and Parsers**: This approach is very easy to implement, only specifications of the lexical analyzer or parser need to be written. The **lex tool** produces the corresponding C code. But this approach is not very efficient which can sometimes be an issue. We can also use a hybrid approach wherein we use high level languages or efficient tools to produce the basic code and if there are some hot-spots (some functions are a bottleneck) then they can be replaced by fast and efficient assembly language routines.

Construct a lexical analyzer

- Allow white spaces, numbers and arithmetic operators in an expression
- Return tokens and attributes to the syntax analyzer
- A global variable tokenval is set to the value of the number
- Design requires that
    - A finite set of tokens be defined
    - Describe strings belonging to each token

We now try to construct a lexical analyzer for a language in which white spaces, numbers and arithmetic operators in an expression are allowed. From the input stream, the lexical analyzer recognizes the tokens and their corresponding attributes and returns them to the syntax analyzer. To achieve this, the function returns the corresponding token for the lexeme and sets a global variable, say ***tokenval*** , to the value of that token. Thus, we must define **a finite set of tokens** and specify the strings belonging to each token. We must also keep a **count of the line number** for the purposes of reporting errors and debugging. We will have a look at a typical code snippet, which implements a lexical analyzer in the subsequent page.

Lexical Analysis Code Example

```
#include <stdio.h>
#include <ctype.h>
int lineno = 1;
int tokenval = NONE;
int lex() {
        int t;
        while (1) {
        t = getchar ();
        if (t = = ' ' || t = = '\t');
        else if (t = = '\n')
                lineno = lineno + 1;
        else if (isdigit (t) ) {
            tokenval = t - '0' ;
        t = getchar ();
      while (isdigit(t)) {
    tokenval = tokenval * 10 + t - '0' ;
    t = getchar();  }
ungetc(t,stdin);
return num; }
else {
tokenval = NONE;return t;
}
}
```

A simple implementation of lex( ) analyzer to eliminate **white space** and **collect numbers** is shown. Every time the body of the while statement is executed, a character is read into t. If the character is a blank (written ' ') or a tab (written '\t'), then no token is returned to the parser; we merely go around the while loop again. If a character is a new line (written '\n'), then a global variable "lineno" is incremented, thereby keeping track of line numbers in the input, but again no token is returned. Supplying a line number with the error messages helps pin point errors. The code for reading a sequence of digits is on lines 11-19. The predicate isdigit(t) from the include file <ctype.h> is used on lines 11 and 14 to determine if an incoming character t is a digit. If it is, then its integer value is given by the expression t-'0' in both ASCII and EBCDIC. With other character sets, the conversion may need to be done differently.

Problems

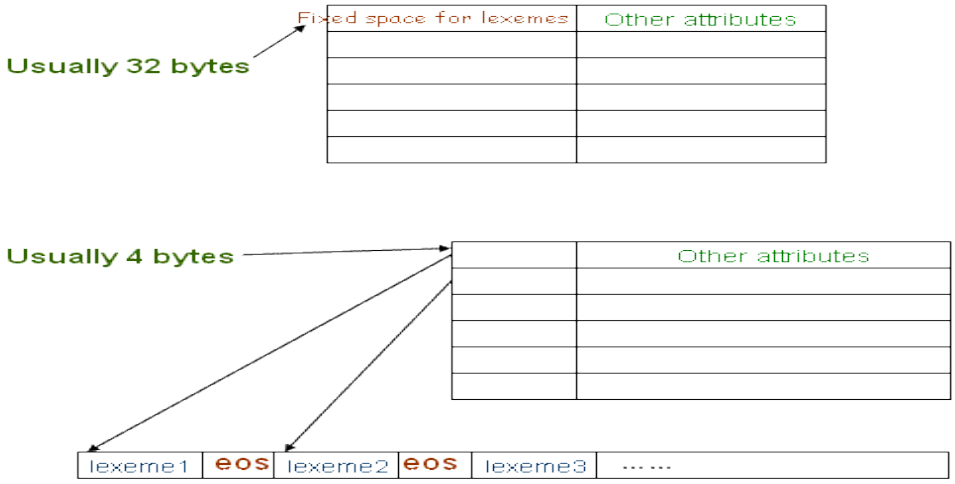
- Scans text character by character
- Look ahead character determines what kind of token to read and when the current token ends
- First character cannot determine what kind of token we are going to read
- The problem with lexical analyzer is that the input is scanned character by character.
- Now, its not possible to determine by only looking at the first character what kind of token we are going to read since it might be common in multiple tokens. We saw one such an example of > and >= previously.
- So one needs to use a lookahead character depending on which one can determine what kind of token to read or when does a particular token end. It may not be a punctuation or a blank but just another kind of token which acts as the word boundary.
- The lexical analyzer on the above code we used a function **ungetc**( ) to push lookahead characters back into the input stream. Because a large amount of time can be consumed moving characters, there is actually a lot of overhead in processing an input character. To reduce the amount of such overhead involved, many specialized buffering schemes have been developed and used.

Symbol Table

- Stores information for subsequent phases
- Interface to the symbol table
    - Insert(s,t): save lexeme s and token t and return pointer
    - Lookup(s): return index of entry for lexeme s or 0 if s is not found

Implementation of symbol table

- Fixed amount of space to store lexemes. Not advisable as it waste space.
- Store lexemes in a separate array. Each lexeme is separated by eos. Symbol table has pointers to lexemes.
- A data structure called **symbol table** is generally used to store information about various source language constructs. Lexical analyzer stores information in the symbol table for the subsequent phases of the compilation process. The symbol table routines are concerned primarily with saving and retrieving lexemes. When a lexeme is saved, we also save the token associated with the lexeme. As an interface to the symbol table, we have two functions
- Insert(s , t): Saves and returns index of new entry for string s , token t .
- -Lookup(s): Returns index of the entry for string s , or 0 if s is not found.
- Next, we come to the issue of implementing a symbol table.
- The symbol table access should not be slow and so the data structure used for storing it should be efficient. However, having a fixed amount of space to store lexemes is not advisable because a fixed amount of space may not be large enough to hold a very long identifier and may be wastefully large for a short identifier, such as i .
- An alternative is to store lexemes in a separate array. Each lexeme is terminated by an **end-of-string**, denoted by EOS, that may not appear in identifiers. The symbol table has pointers to these lexemes.



- Here, we have shown the two methods of implementing the symbol table which we discussed in the previous page in detail.
- As, we can see, the first one which is based on allotting fixed amount space for each lexeme tends to waste a lot of space by using a fixed amount of space for each lexeme even though that lexeme might not require the whole of 32 bytes of fixed space.
- The second representation which stores pointers to a separate array, which stores lexemes terminated by an EOS, is a better space saving implementation.
- Although each lexeme now has an additional overhead of five bytes (four bytes for the pointer and one byte for the EOS).
- Even then we are saving about 70% of the space which we were wasting in the earlier implementation. We allocate extra space for 'Other Attributes' which are filled in the later phases.

How to handle keywords?

- Consider token DIV and MOD with lexemes **div** and **mod**.
- Initialize symbol table with insert ("div" , DIV) and insert("mod" , MOD).
- Any subsequent lookup returns a nonzero value, therefore, cannot be used as an identifier .
- To handle keywords, we consider the keywords themselves as **lexemes**. We store all the entries corresponding to keywords in the symbol table while initializing it and do lookup whenever we see a new lexeme.
- Now, whenever a lookup is done, if a nonzero value is returned, it means that there already exists a corresponding entry in the Symbol Table.

- So, if someone tries to use a keyword as an identifier, it will not be allowed as an identifier with this name already exists in the Symbol Table.
- For instance, consider the tokens *DIV* and *MOD* with lexemes "div" and "mod".
- We initialize symbol table with insert ("div", DIV) and insert ("mod", MOD). Any subsequent lookup now would return a nonzero value, and therefore, neither "div" nor "mod" can be used as an identifier.

Difficulties in design of lexical analyzers

- Is it as simple as it sounds?
- Lexemes in a fixed position. **Fix format** vs. **free format languages**
- The design of a lexical analyzer is quite complicated and not as simple as it looks.
- There are several kinds of problems because of all the different types of languages we have. Let us have a look at some of them. For example:
- 1. **We have both fixed format and free format languages** - A lexeme is a sequence of character in source program that is matched by pattern for a token.
- **FORTRAN** has lexemes in a **fixed position**. These white space and fixed format rules came into force due to punch cards and errors in punching. Fixed format languages make life difficult because in this case we have to look at the position of the tokens also.
- 2. **Handling of blanks** - It's of our concern that how do we handle blanks as many languages (like Pascal, FORTRAN etc) have significance for blanks and void spaces.
- When more than one pattern matches a lexeme, the lexical analyzer must provide additional information about the particular lexeme that matched to the subsequent phases of the lexical analyzer.
    - In Pascal blanks separate identifiers.
    - In FORTRAN blanks are important only in literal strings. For example, the variable ”**counter** " is same as " **count er** ".
- For example DO 10 I = 1.25 and DO 10 I = 1,25
- The first line is a variable assignment DO10I = 1.25.
- The second line is the beginning of a **Do** loop.
- In such a case, we might need an arbitrary long lookahead. Reading from left to right, we cannot distinguish between the two until the “, " or " . " is reached.

How to specify tokens?

The various issues which concern the specification of tokens are:

1. How to describe the complicated tokens like e0 20.e-01 2.000

2. How to break into tokens from input statements like **if** (x==0) a = x << 1; **iff** (x==0) a = x < 1;

3. How to break the input into tokens efficiently?

There are the following problems that are encountered:

- Tokens may have similar prefixes

- Each character should be looked at only once

How to describe tokens?

- Programming language tokens can be described by regular languages

Regular languages

- Are easy to understand
- There is a well understood and useful theory
- They have efficient implementation

- Regular languages have been discussed in detail in the " **Finite state automata and Computation theory**" course.

  Here we address the problem of describing tokens. **Regular expression** is an important notation for specifying patterns. Each pattern matches a set of strings, so regular expressions will serve as names for set of strings.

  Programming language tokens can be described by **regular languages**. The specification of regular expressions is an example of an recursive definition. Regular languages are easy to understand and have efficient implementation.

  The theory of regular languages is well understood and very useful. There are a number of algebraic laws that are obeyed by regular expression, which can be used to manipulate regular expressions into equivalent forms.

Operations on languages

- . L U M = {s | s is in L or s is in M}
- . LM = {st | s is in L and t is in M}
- The various operations on languages are:
- Union of two languages L and M written as L U M = {s | s is in L or s is in M}
- Concatenation of two languages L and M written as LM = {st | s is in L and t is in M}
- The Kleene Closure of a language L written as

$L^* = \text{Union of } L^i \text{ such that } 0 \le i \le \infty \text{ where } L^0 = \epsilon \text{ and } L^i = L^{i-1} L$

For Example

- Let L = {a, b, .., z} and D = {0, 1, 2, . 9} then
- LUD is a set of letters and digits
- LD is a set of strings consisting of a letter followed by a digit
- L* is a set of all strings of letters including Є(epsilon)
- L(LUD)* is a set of all strings of letters and digits beginning with a letter
- D + is a set of strings of one or more digits

Example:

Let L be a the set of alphabets defined as L = {a, b, .., z} and D be a set of all digits defined as D = {0, 1, 2, .., 9}. We can think of L and D in two ways. We can think of L as an alphabet consisting of the set of lower case letters, and D as the alphabet consisting of the set the ten decimal digits. Alternatively, since a symbol can be regarded as a string of length one, the sets L and D are each finite languages. Here are some examples of new languages created from L and D by applying the operators defined previously.

- Union of L and D, L U D is the set of letters and digits.
- Concatenation of L and D, LD is the set of strings consisting of a letter followed by a digit.
- The Kleene closure of L, L* is a set of all strings of letters including ?.
- L(LUD)* is the set of all strings of letters and digits beginning with a letter.
- D+ is the set of strings one or more digits.

Let S be a set of characters. A language over S is a set of strings of characters belonging to S . A regular expression is built up out of simpler regular expressions using a set of defining rules. Each regular expression r denotes a language L(r). The defining rules specify how L(r) is formed by combining in various ways the languages denoted by the sub expressions of r . Following are the rules that define the regular expressions over S :

- € is a regular expression that denotes { € }, that is, the set containing the empty string.
- If **a** is a symbol in S then **a** is a regular expression that denotes { a } i.e., the set containing the string a . Although we use the same notation for all three, technically, the regular expression a is different from the string a or the symbol a . It will be clear from the context whether we are talking about a as a regular expression, string or symbol.

Suppose r and s are regular expressions denoting the languages L(r) and L(s). Then,

- (r)|(s) is a regular expression denoting L(r) U L(s).
- (r) (s) is a regular expression denoting L(r) L(s).
- (r)* is a regular expression denoting (L(r))*.
- (r) is a regular expression denoting L(r).

Let us take an example to illustrate: Let S = {a, b}.

1. The regular expression a|b denotes the set {a,b}.
2. The regular expression (a | b) (a | b) denotes {aa, ab, ba, bb}, the set of all strings of a's and b's of length two. Another regular expression for this same set is aa | ab | ba | bb.
3. The regular expression a* denotes the set of all strings of zero or more a's i.e., { ? , a, aa, aaa, .}.
4. The regular expression (a | b)* denotes the set of all strings containing zero or more instances of an a or b, that is, the set of strings of a's and b's. Another regular expression for this set is (a*b*)*.
5. The regular expression a | a*b denotes the set containing the string a and all strings consisting of zero or more a's followed by a b.

•If two regular expressions contain the same language, we say r and s are equivalent and write r = s. For example, (a | b) = (b | a).

Precedence and associativity

- *, **concatenation**, and | are left associative
- * has the highest precedence
- **Concatenation** has the second highest precedence and | has the lowest precedence

How to specify tokens

Regular definitions

- Let r_i be a regular expression and d_i be a distinct name
- Regular definition is a sequence of definitions of the form
  $d_1 \rightarrow r_1$
  $d_2 \rightarrow r_2$
  ....
  $d_n \rightarrow r_n$
- Where each r_i is a regular expression over $\Sigma \cup \{d_1, d_2, \ldots, d_{i-1}\}$

If S is an alphabet of basic symbols, then a regular definition is a sequence of definitions of the form

$d_1 \longrightarrow r_1$
$d_2 \longrightarrow r_2$
.....
$d_n \longrightarrow r_n$

where each d_i is a distinct name, and each r_i is a regular expression over the symbols in $\Sigma \cup \{d_1, d_2, \ldots, d_{i-1}\}$ i.e. the basic symbols and the previously defined names. By restricting each r_i to symbols of S and the previously defined names, we can construct a regular expression over S for any r_i by repeatedly replacing regular-expression names by the expressions they denote.

Lexical analyzer generator

- Input to the generator
  - List of regular expressions in priority order
  - Associated actions for each of regular expression (generates kind of token and other book keeping information)
- Output of the generator
  - Program that reads input character stream and breaks that into tokens
  - Reports lexical errors (unexpected characters), if any

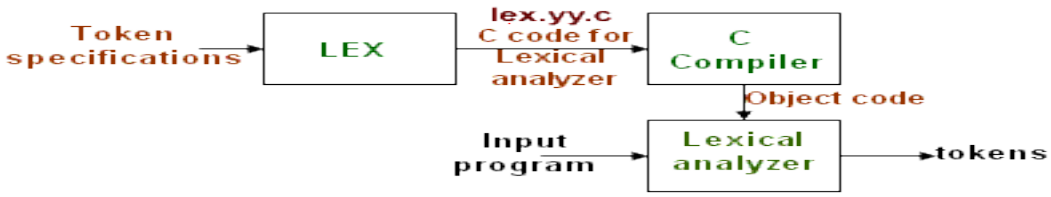
We assume that we have a specification of lexical analyzers in the form of regular expression and the corresponding action parameters.

Action parameter is the program segments that is to be executed whenever a lexeme matched by regular expressions is found in the input. So, the input to the generator is a list of regular expressions in a priority order and associated actions for each of the regular expressions. These actions generate the kind of token and other book keeping information.

Our problem is to construct a recognizer that looks for lexemes in the input buffer. If more than one pattern matches, the recognizer is to choose the longest lexeme matched. If there are two or more patterns that match the longest lexeme, the first listed matching pattern is chosen.

So, the output of the generator is a program that reads input character stream and breaks that into tokens. It also reports in case there is a lexical error i.e. either unexpected characters occur or an input string doesn't match any of the regular expressions.

LEX: A lexical analyzer generator



In this section, we consider the design of a software tool that automatically constructs the lexical analyzer code from the LEX specifications. LEX is one such lexical analyzer generator which produces C code based on the token specifications. This tool has been widely used to specify lexical analyzers for a variety of languages. First, a specification of a lexical analyzer is prepared by creating a program lex.l in the lex language. Then, the lex.l is run through the Lex compiler to produce a C program lex.yy.c . The program lex.yy.c consists of a tabular representation of a transition diagram constructed from the regular expressions of the lex.l, together with a standard routine that uses the table to recognize lexemes.

How does LEX work?

- Regular expressions describe the languages that can be recognized by finite automata
- Translate each token regular expression into a non deterministic finite automaton (NFA)
- Convert the NFA into an equivalent DFA
- Minimize the DFA to reduce number of states
- Emit code driven by the DFA tables
- In this section, we will describe the working of lexical analyzer tools such as LEX. LEX works on some fundamentals of regular expressions and NFA - DFA.
- First, it reads the regular expressions which describe the languages that can be recognized by finite automata. Each token regular expression is then translated into a corresponding non-deterministic finite automaton (NFA). The NFA is then converted into an equivalent deterministic finite automaton (DFA). The DFA is then minimized to reduce the number of states. Finally, the code driven by DFA tables is emitted.